

Simulación de Ecosistema con OpenMP

Javier España, José Mérida, Angel Esquit
Computación Paralela y Distribuida

13 de agosto de 2026

Este proyecto implementa una simulación de ecosistema sobre una grilla 2D (plantas, herbívoros, carnívoros) que avanza en *ticks* discretos, paralelizada con OpenMP.

Repositorio: <https://github.com/AngelEsquit/MiniPRY-CPD>

1. Reglas del Ecosistema

Cada celda de la grilla guarda:

```
typedef struct {  
    int type;      // EMPTY, PLANT, HERBIVORE, CARNIVORE  
    int energy;  
    int starve;    // ticks consecutivos sin comer  
    int age;  
} Cell;
```

Plantas. Cada tick, con 30 % de probabilidad, una planta se expande a una celda vacía adyacente al azar; la planta original no desaparece, sigue en su celda y crece un brote nuevo en la vecina. Muere únicamente si es consumida por un herbívoro.

Herbívoros. Cada tick, un herbívoro pondera sus hasta 4 celdas vecinas transitables (vacías, o con una planta que puede comer) como suma ponderada de:

- + $5 \times$ distancia de ese vecino al carnívoro más cercano
- $2 \times$ distancia de ese vecino a la planta más cercana
- + 20 si ese vecino tiene una planta (bono fijo por comer)

y se mueve al vecino de mayor puntaje (se queda quieto si ninguno es transitable). Es un trade-off continuo entre huir del carnívoro más cercano y acercarse a la planta más cercana, así podemos emular de manera más cercana un comportamiento adecuado. Al comer gana energía, si llega a $\geq 2 \times$ su energía inicial, puede reproducirse (deja una cría en la celda que acaba de desocupar). Muere por no comer, por vejez, o por ser comido.

Carnívoros. Misma estructura, pero sin comportamiento de huida. Si hay un herbívoro adyacente lo cazan y si no, caminan hacia el herbívoro más cercano. Si no hay ninguno cerca, deambulan. La reproducción y la muerte siguen la misma lógica que los herbívoros, con sus propias constantes de energía y hambre.

Competencia entre especies. Las tres especies deciden su movimiento sobre la misma foto congelada de la grilla (nadie ve el resultado de nadie más dentro del mismo tick), así que puede pasar que varias celdas de especies distintas propongan la misma celda destino. Por ejemplo, una planta expandiéndose a una celda vacía y un herbívoro deambulando hacia esa misma celda. Eso se resuelve con una prioridad fija por tipo de movimiento (más detalle en la Sección 2.6): cazar (20) > huir (15) > comer planta (10) > deambular/expandirse (5 / 1).

2. Funcionamiento

2.1. Organización del Código

El código está separado por responsabilidad en `src/`:

<code>src/</code>	
<code>internal.h</code>	Estado y helpers compartidos (privado)
<code>main.c</code>	CLI
<code>core/</code>	Estado, mapas de distancia, orquestacion del tick, apply
<code>species/</code>	Una decision de movimiento por especie
<code>sync/</code>	Resolucion de conflictos de destino
<code>io/</code>	Salida de consola
<code>util/</code>	RNG

2.2. Estructuras Principales

- `Cell`: el estado de una celda (`type`, `energy`, `starve`, `age`).
- `Move`: la propuesta de movimiento de una celda origen (`dst_i`, `dst_j`, `score`, `valid`), guardada en el arreglo `moves[]`.
- `claim_score[]` / `claim_src[]`: por cada celda destino, el mejor puntaje visto hasta ahora y quién lo propuso. Es el resultado de la resolución de conflictos.
- `moved[]` / `eaten[]`: banderas por celda origen para dos casos especiales (alguien que se movió de verdad, alguien que fue cazado en su propia celda) que la resolución por destino, por sí sola, no cubre.
- `Birth`: una cría en cola, con su celda y su tipo (`births[]`, `birth_count`).
- `dist_to_plant[]`, `dist_to_herbivore[]`, `dist_to_carnivore[]`: los mapas de distancia usados por herbívoros y carnívoros para decidir hacia dónde moverse.

2.3. Ciclo del Tick

Todo tick lee de `grid_cur` y escribe en `grid_next`, intercambiando los punteros recién al final (doble buffer). Si todos los hilos leyeran y escribieran la misma grilla a la vez, el resultado dependería del orden de ejecución de los hilos, que en OpenMP no está definido. Con doble buffer, cada ronda paralela lee de un array que nadie escribe y escribe en uno que nadie lee, así que no hace falta ningún lock. El mismo truco se usa dentro de cada mapa de distancia (Sección 2.4).

1. Se recalculan tres mapas de distancia (a la planta, herbívoro y carnívoro más cercanos) desde `grid_cur`.
2. Cada especie propone, en paralelo, un movimiento por celda hacia `moves[]`.

3. Se resuelven los conflictos de destino, llenando `claim_src[]`, `moved[]` y `eaten[]`.
4. Se aplica el resultado a `grid_next`, usando esas tres estructuras.
5. Se intercambian `grid_cur` y `grid_next` y se repite.

2.4. Mapas de Distancia

Calcular "la planta más cercana" para cada celda es un BFS multi-fuente. En vez de una cola compartida (su propio problema de sincronización), se calcula por relajación en rondas, con el mismo truco de doble buffer:

```
buf_a[k] = (grid_cur[k].type == seed_type) ? 0 : DIST_INF;
for (round = 0; round < MAX_RELAX_RADIUS; round++) {
    #pragma omp parallel for collapse(2)
    for (i, j) {
        best = buf_a[IDX(i,j)];
        for cada vecino: best = min(best, buf_a[vecino] + 1);
        buf_b[IDX(i,j)] = best;
    }
    swap(buf_a, buf_b);
}
```

Por ronda, cada hilo procesa una celda: lee su propia distancia y la de sus 4 vecinos en `buf_a`, y escribe un único valor en `buf_b`. Como todos leen de `buf_a` y nadie lo escribe esa ronda, no hace falta coordinación entre hilos hasta el barrier implícito al final del `parallel for`, que marca cuándo la siguiente ronda puede arrancar.

Las rondas se acotan a `MAX_RELAX_RADIUS = 8`: a un organismo solo le importa la dirección hacia el objetivo, no la distancia exacta en toda la grilla.

2.5. Lista de Movimientos

Cada celda origen escribe su movimiento propuesto en `moves[IDX(i,j)]`, su propio índice y ningún otro. Como ningún par de hilos escribe jamás la misma posición, esta etapa no necesita sincronización.

2.6. Resolución de Conflictos

El único conflicto real es cuando dos celdas origen (de cualquier especie) proponen la misma celda destino. Se resuelve con un `omp_lock_t` por celda de la grilla:

```
omp_set_lock(&cell_locks[dst]);
if (moves[k].score > claim_score[dst]) {
    claim_score[dst] = moves[k].score;
    claim_src[dst] = k;
}
omp_unset_lock(&cell_locks[dst]);
```

Cada lock protege una sola celda, así que dos hilos yendo a destinos distintos nunca se bloquean entre sí.

La prioridad por score cubre exclusivamente empates sobre una misma celda destino; por las reglas de movimiento, la celda de un herbívoro vivo solo puede ser destino de un carnívoro cazándolo, así que ahí el único empate posible es entre dos carnívoros cazando al mismo herbívoro.

Después de resolver corren dos pasadas cortas de etiquetado, sin locks (cada una escribe un índice distinto por iteración, libre de carrera): `moved[]` marca qué celdas se vaciaron de verdad, para no duplicar a alguien que se movió como si además se hubiera quedado quieto; `eaten[]` marca herbívoros cazados en su propia celda de origen, cubriendo el caso en el que ese mismo herbívoro también ganó, en una celda destino distinta, su propio movimiento hacia otra parte. `core/apply.c` revisa `eaten[]` antes de colocar a cualquier ganador de movimiento.

2.7. Nacimientos

Una cría se coloca en la celda que su padre acaba de desocupar, una celda que también se procesa, en la misma pasada paralela, como destino de otra celda cualquiera. Escribirla directamente ahí sería una carrera, así que cada nacimiento reserva un lugar en una cola con un contador atómico (`#pragma omp atomic capture`) y se coloca en una pasada aparte, después de que la pasada principal termina.

2.8. Números Aleatorios

Cada hilo tiene su propia semilla (`seeds[tid]`) y usa `rand_r`, reentrante, para no competir por el estado global de `rand()`.

3. Compilación y Ejecución

```
make                # genera build/*.o y el binario ./ecosystem
make clean          # borra build/ y el binario
./ecosystem <filas> <cols> <ticks> <hilos>
./ecosystem 60 60 100 8
```

La población inicial se deriva del tamaño de la grilla: 30 % plantas, 10 % herbívoros, 5 % carnívoros. Cada 10 ticks (y en los ticks 1 y 5) se imprime el conteo de población por especie y, si la grilla es de 30x30 o menos, un mapa de caracteres de la distribución.

4. Resultados

En corridas de prueba (60x60, 40 ticks) se observa el patrón esperado de un sistema depredador-presa: plantas fluctuando con la presión de los herbívoros, herbívoros creciendo con la comida disponible y cayendo con la caza, carnívoros decayendo si la población de herbívoros colapsa. No se observó duplicación de organismos ni poblaciones que excedan la capacidad de la grilla en ninguna corrida.

Tiempo total de simulación variando el número de hilos (grilla 200x200, 50 ticks, máquina con 20 núcleos lógicos):

Hilos	Tiempo total (s)	Tiempo/tick (ms)	Speedup
1	0.2138	4.28	1.00×
2	0.1297	2.59	1.65×
4	0.0878	1.76	2.44×
8	0.0860	1.72	2.49×

El speedup se aplana entre 4 y 8 hilos: cada tick lanza más de veinte regiones paralelas distintas (8 rondas $\times$ 3 mapas de distancia, más decisión/resolución/aplicación), y cada una paga un costo fijo de arranque de hilos que, en una grilla de este tamaño, empieza a pesar más que el trabajo ganado por hilo adicional. Para grillas más grandes se esperaría que ese punto se corra hacia más hilos.